<PLACE_TEXT> |
<PLACE_TEXT>, + space + <PLACE_NAME>
]

where:

space = U+0020, the Space character

The jurisdictional name of the place where the event took place. Jurisdictions are separated by a comma and space combination. For example: "Cove, Cache, Utah, United States of America".
No part of the place name may be replaced by an abbreviation. Place names are not terminated by a full stop or anything else.

PLACE_PHONETIC:= **{Size=1:120}**

The phonetic variation of the place name is written in the same form as was the place name used in the superior <PLACE_NAME> primitive, but phonetically written using the method indicated by the subordinate <PHONETISATION_METHOD> value. For example if hiragana was used to provide a phonetic transcription of a name written in kanji, then the <PHONETISATION_METHOD> value would indicate kana. (See PHONETISATION_METHOD, page 100.)

PLACE_ROMANISED:= **{Size=1:120}**

The romanised transcription of the place name is written in the same form prescribed for the place name used in the superior <PLACE_NAME> context. The method used to romanise the name is indicated by the line value of the subordinate <ROMANISATION_METHOD>, for example if romaji was used to provide a transcription of a place name written in kanji, then the <ROMANISATION_METHOD> subordinate to the ROMN tag would indicate 'romaji' (See ROMANISATION_METHOD.), page 105

PLACE_TEXT:= **{Size=1:120}**

<TEXT> excluding any commas.

A fully specified place name exists of several parts, from place name up to country, separated by comma & space combinations, <PLACE_TEXT> is one such place name part. As place name parts are separated by commas, including any commas within a place name part would create confusion, so they have to be left out.

Example

An example of a place name that includes a comma is "De Mijl, Krabbe en Nadort" (without quotes), which existed in the province of Noord-Brabant in the Netherlands. That place should be specified as "De Mijl Krabbe en Nadort, Noord-Brabant, Netherlands" (without quotes).

Violation of Place Name Guidelines

The demand to leave out commas violates basic place name guidelines. *The* basic guideline is to enter place name parts *as-is*, and that means that you should always use full names, never abbreviations, and always use just the actual name, without anything added to it, and without leaving anything out.

The GEDCOM specification demands one exception to this rule: leave commas out of place names. The GEDCOM specification makes that demand because GEDCOM cannot support place names that include commas correctly.
The FamilySearch GEDCOM creators probably thought such place names do not exist, and later discovered that such names do exist. You should include such place

names without their comma, just as the FamilySearch GEDCOM specification demands.

References

Place Name Standardisation Basics

POSSESSIONS:= **{Size=1:248}**

A list of possessions (real estate or other property) belonging to this individual.

PRODUCT_VERSION_NUMBER:= **{Size=3:15}**

MMM + dot + mmm [+ dot + rrr [+ dot + bbb]]

where:

MMM = 1 through 3 digits; the major version number
mmm = 1 through 3 digits; the minor version number
rrr = 1 through 3 digits; the revision number
bbb = 1 through 3 digits; the build number
dot = U+002E, the Full Stop character

The version of the product that created the GEDCOM file.
The product version number is controlled by the developers of the product, but *must* be in this format, and *should* comply with common software industry version number practices briefly mentioned here.

The product version number *must* be provided through <PRODUCT_VERSION_NUMBER> and should not occur anywhere else. Specifically, the version number *must not* be used as part of the product name, but it is allowed to use a *marketing version* in the product name; e.g. “Family Tree Maker 2017” is a marketing version of the Family Tree Maker product. System version and build information is conveyed through HEAD.SOUR.VER and through HEAD.SOUR.VERS only.

Four Values

A product version consists of at least two and most four dot-separated values in *major.minor.revision.build* format. The four parts of the *major.minor.revision.build* format are:

The four parts of the *major.minor.revision.build* format are:

major = the major version number; starts at 0 (zero).
minor = the minor version number; starts at 0 for each new *major* version.
revision = the revision number; starts at 0 for each new *major.minor* version.
build = the build number for *major.minor.revision*.

Dot-Separated Values

The full stops used in version numbers are known as dots, but pronounced as “point”. A version number never begins or ends with a dot, it always begins and ends with a digit. Values used must appear in the order shown. The major and minor version number are both mandatory. The minor version number must included, even if it is zero.
The revision number may be left off when it zero.
The build number is optional too, but may only be included if all the other three values are included. All missing values must be assumed to be zero.

The *major* version number signals a major new release, often available as an paid *upgrade*.
The *minor* version number version number signals a significant update of the same release,

often available as a free *update*. The *revision* number signals a lesser update, typically available as a free *hotfix*.
While the first public release of a product may have version number 1.0, earlier development releases should have version numbers too.

Each value consists at most 3 digits. No other characters are allowed. Zeroes are allowed, leading zeroes are not; `0.99` is a valid version number, and so is `1.0`, but `1.01` is not. Trailing zeroes must not be left out and aren't implied; version `2.1` and version `2.10` are two different versions, and version `2.1` comes before version `2.9`. The minimum version number is `0.1`. The maximum version number is `999.999.999.999`.

Build Number

The build number is a number that is different for each build of the same *major.minor.revision* version, and may indicate the platform the code was build for. Genealogy software developers that wish to distinguish between 32-bit and 64-bit builds *must not* do so by messing with the *major.minor.revision* version, but may opt to use build number `32` and build number `64` to indicate the 32-bit and 64-bit builds respectively. Systems that use the build number to indicate 32-bit and 64-bit build must always specify all four values.

Many developers use an ever-increasing build number for their product, and can continue doing so, but that product build number must not be used as part of the <PRODUCT_VERSION_NUMBER>. Developers that wish to include their product build number in a GEDCOM file may use it as part of the product name.

Microsoft uses the words build and revision differently; if your genealogy system is a Microsoft .NET application, do not worry too much about it, simply use *major.minor.build.revision* as defined within .NET, and you should be fine.

PUBLICATION_DATE:= {Size=10:11}

<DATE_EXACT>
The date this source was published or created.

RECEIVING_SYSTEM_NAME:= {Size=1:20}

The system identifier of the system expected to process the GEDCOM file.

GEDCOM Interpretation

All GEDCOM writers must default the HEAD.DEST value to their own system identifier.
The HEAD.DEST value specifies the interpretation of any GEDCOM extensions. In a typical GEDCOM file the HEAD.SOUR and HEAD.DEST values are identical; and the meaning of that typical situation is that the GEDCOM writer used its own GEDCOM extensions, if any.
A GEDCOM 5.5.5 writer should only specify another system identifier than its own, when the GEDCOM file is intended for import by a specific system, and the GEDCOM writer produces GEDCOM extensions for that system *instead of* its own. This is not a generally recommended practice, but can be handy when producing GEDCOM files for some particular third party system. While the HEAD.DEST value may be a system identifier for an old and no longer supported system, it may only be a system identifier for a system known to support GEDCOM 5.5.5.

Nonsense Values

GEDCOM specifications before the GEDCOM 5.5.1 Annotated Edition (2018 CE) failed to clearly specify the usage of the HEAD.DEST value. Many GEDCOM 5.5 and 5.5.1 files contain nonsense values such as `ANY`, `GEDCOM`, `GEDCOM55`, and `Other`, despite the fact

that the spec demands the value to be a system identifier.

GEDCOM 5.5.5 demands that the HEAD.DEST value is a real system identifier. Known nonsense values must not be used as system identifiers.

A GEDCOM 5.5.5 validator must issue a strong warning when the HEAD.SOUR and HEAD.DEST values aren't identical, and a fatal error when it recognises a known nonsense value.
A GEDCOM validator should also issue an error when it recognises a system identifier for a system that does not support GEDCOM 5.5.5, and a strong warning when it does not recognise the system identifier used (probable typo).

More information about system identifiers is provided in Chapter 4 GEDCOM System Identifiers, page 117.

RELATION_IS_DESCRIPTOR:= {Size=1:25}

A word or phrase that states object 1's **relation** is object 2. For example you would read the following as “Joe Jacob's high-school geography teacher is the individual identified by @I551@”:

0 INDI
1 NAME Joe /Jacob/
1 ASSO @I551@
2 RELA high-school geography teacher

The <RELATION_IS_DESCRIPTOR> is the ASSO.RELA line value, which provides a description of the association created by the <<ASSOCIATION_STRUCTURE>> (ASSO record) it is part of.
Notice that the <RELATION_IS_DESCRIPTOR> *does not* provide any predefined values, but is a free-form text field; by the user, for the user.

Applications cannot and should not try to rely on the user or other applications using particular values, or any particular value having any particular meaning.
That said, some common values are: `godfather`, `godmother`, `attendant`, `informant`, `witness` and `Other`, but you may encounter many different values, such as `godparent`, `officiating priest`, including illegal values such as `brother`, or `Grand pere maternel` (maternal grandfather in French).

RELIGIOUS_AFFILIATION:= {Size=1:90}

A name of the religion with which this person, event, or record was affiliated.

RESPONSIBLE_AGENCY:= {Size=1:120}

The organization, institution, corporation, person, or other entity that has responsibility for the associated context. For example, an employer of a person of an associated occupation, or a church that administered rites or events, or an organization responsible for creating and/or archiving records.

ROLE_DESCRIPTOR:= {Size=1:25}

A word or phrase that identifies a person's role in an event being described. This should be the same word or phrase, and in the same language, that the recorder used to define the role in the actual record.

ROLE_IN_EVENT:= {Size=3:27}

[CHIL | HUSB | WIFE | MOTH | FATH | SPOU | **(** <ROLE_DESCRIPTOR> **)**]

Indicates what role this person played in the event that is being cited in this context. For example, if you cite a child's birth record as the source of the mother's name, the value for this field is "MOTH". If you describe the groom of a marriage, the role is "HUSB". If the

role is something different than one of the six relationship role tags listed above then enclose the role name within matching parentheses.

The six predefined values must not be within parentheses, but that additional, system-defined or user-defined values must be. It is an error to put any of the predefined values within parentheses, or not use parentheses for other values.

ROMANISATION_METHOD:= {Size=5:30}

[<user defined> | pinyin | romaji | wadegiles]
Indicates the method used in transforming the text to a romanised transcription.

SCHOLASTIC_ACHIEVEMENT:= {Size=1:248}

A description of a scholastic or educational achievement or pursuit.

SEX_VALUE:= {Size=1:1}

A code that indicates the sex of an individual:

M = Male
F = Female
X = Intersex
U = Unknown (not found yet)
N = Not recorded

`U` is the default value.

The values `U` and `N` are mostly used for stillborn children, but must be used for every situation where the sex is unknown (yet).
The difference between the two values is significant. The value `U` merely says that you do not know the sex (yet), because you did not find or consult a record. The value `N` says that the record did not record a sex, or that you are sure that there is no record.

The current GEDCOM specification allows just one sex per individual. A future GEDCOM version should provide the ability to specify different values for the biological, official and legal sex. The current field should be used to record the official sex, i.e. the sex on the birth record.

SOURCE_CALL_NUMBER:= {Size=1:120}

An identification or reference description used to file and retrieve items from the holdings of a repository.

SOURCE_DESCRIPTIVE_TITLE:= {Size=1:4095}

The title of the work, record, or item and, when appropriate, the title of the larger work or series of which it is a part.

For a *published* work, a book for example, might have a title plus the title of the series of which the book is a part. A magazine article would have a title plus the title of the magazine that published the article.

For An *unpublished* work, such as:

- **!** A letter might include the date, the sender, and the receiver.
- **!** A transaction between a buyer and seller might have their names and the transaction date.
- **!** A family Bible containing genealogical information might have past and present owners and a physical description of the book.
- **!** A personal interview would cite the informant and interviewer.

SOURCE_FILED_BY_ENTRY:= **{Size=1:60}**

This entry is to provide a short title used for sorting, filing, and retrieving source records.

SOURCE_JURISDICTION_PLACE:= **{Size=1:120}**

<PLACE_NAME>
The name of the lowest jurisdiction that encompasses all lower-level places named in this source. For example, "Oneida, Idaho" would be used as a source jurisdiction place for events occurring in the various towns within Oneida County. "Idaho" would be the source jurisdiction place if the events recorded took place in other counties as well as Oneida County.

SOURCE_MEDIA_TYPE:= **{Size=1:15}**

[audio | book | card | electronic | fiche | film | magazine | manuscript | map | newspaper | photo | tombstone | video]

A code, selected from one of the media classifications choices above, that indicates the type of material in which the referenced source is stored.

SOURCE_ORIGINATOR:= **{Size=1:255}**

The person, agency, or entity who created the record. For a published work, this could be the author, compiler, transcriber, abstractor, or editor. For an unpublished source, this may be an individual, a government agency, church organization, or private organization, etc.

SOURCE_PUBLICATION_FACTS:= **{Size=1:4095}**

When and where the record was created. For published works, this includes information such as the city of publication, name of the publisher, and year of publication.

For an unpublished work, it includes the date the record was created and the place where it was created. For example, the county and state of residence of a person making a declaration for a pension or the city and state of residence of the writer of a letter.

SUBMITTER_NAME:= **{Size=1:60}**

The name of the submitter formatted for display and address generation.

SYSTEM_ID:= **{Size=1:20}**

A system identification name. This name must be unique for each system (product), different from any other system. The name may include spaces, and is *not* restricted to ASCII characters.

The system identifier occurs as a line value of the mandatory HEAD.SOUR and HEAD.DEST records in the GEDCOM header.

TEXT:= **{Size=1:32767}**

A Unicode string.
Some rules and restrictions apply, see line_char definition in the GEDCOM grammar.

TEXT_FROM_SOURCE:= **{Size=1:32767}**

<TEXT>
A verbatim copy of any description contained within the source. This indicates notes or text that are actually contained in the source document, not the submitter's opinion about the source. This should be, from the evidence point of view, "what the original record keeper said" as opposed to the researcher's interpretation. The word TEXT, in this case, means from the text which appeared in the source record including labels.

TIME_VALUE:= **{Size=7:12}**

hh:mm[:ss[.fs]]
The time of a specific event, usually a computer-timed event, where:

hh = hours on a 24-hour clock, one or two digits; no leading zeroes {0:23}
mm = minutes, a two-digit value *with* leading zeroes {0:59}
ss = seconds, a two-digit value, *with* leading zeroes {0:59}
fs = a two-digit decimal fraction of a second {0:99}

USER_REFERENCE_NUMBER:= **{Size=1:20}**

A user-defined number or text that the submitter uses to identify this record. For instance, it may be a record number within the submitter's automated or manual system, or it may be a page and position number on a pedigree chart.

The <USER_REFERENCE_NUMBER> is the line value of the optional REFN record. The optional REFN occurs in the FAM, INDI, OBJE, NOTE, REPO and SOUR records; so, in all the <LINEAGE_LINKED_RECORD> types.

While the name of this line value strongly suggests that it should be a number, the line value is not limited to digits. It is free-form text that may include any character. It should really have been called <USER_REFERENCE_IDENTIFIER> instead of <USER_REFERENCE_NUMBER>.

The <USER_REFERENCE_NUMBER> allows users to associate any kind of number with these records. These record types allow any number of REFN subrecords, so users can associate as many identifiers to a record as they like.

The user-defined identifiers used need not be unique. The same identifier may be used multiple times, on different records.

The REFN record is for user-defined numbers, the IDNO record is for third-party numbers.

USER_REFERENCE_TYPE:= **{Size=1:40}**

A user-defined definition of the <USER_REFERENCE_NUMBER>.

This value is free-form text, but meant to be a value from a relatively short user-defined list. The <USER_REFERENCE_TYPE> is the line value of the optional REFN.TYPE record; the REFN line value specifies a user reference number, the REFN.TYPE specifies that number's type.
This allows users to classify the reference numbers they use in any way they see fit.

USER_TEXT:= **{Size=1:32767}**

Free-form user text. Comments, opinions.

WHERE_WITHIN_SOURCE:= **{Size=1:248}**

Specific location within the information referenced. For a published work, this could include the volume of a multi-volume work and the page number(s). For a periodical, it could include volume, issue, and page numbers. For a newspaper, it could include a column number and page number. For an unpublished source or microfilmed works, this could be a film or sheet number, page number, frame number, etc. A census record might have an enumerating district, page number, line number, dwelling number, and family number. The data in this field should be in the form of a label and value pair, such as Label1: value, Label2: value, with each pair being separated by a comma. For example, Film: 1234567, Frame: 344, Line: 28.

XREF:= **{Size=3:22}**

Either a pointer or an unique cross-reference identifier. If this element appears before the tag in a GEDCOM line, then it is a cross-reference identifier. If it appears after the tag in a

GEDCOM line, then it is a pointer.

The at signs that delimit a cross-reference identifier or pointer are part of that cross-reference identifier or pointer. The identifying part within the at signs has a minimum length of 1, and maximum length of 20 code units.

Legal Characters

Each <XREF> corresponds to a **xref_ID** in the GEDCOM grammar, the identifying part within the at signs corresponds to a **identifier_string**.
GEDCOM 5.5.1 and earlier allowed almost any character in an identifier string, with a special restriction regarding the use of the number sign (#), and restrictions regarding the use of colon (:) and exclamation mark (!) in an <XREF>.
An identifier string must not start with a number sign (#), as that might create confusion with an escape sequence prefix (@#), but the number sign is no longer legal in identifiers. The Lineage-Linked Form reserved the colon (:) and exclamation mark (!) future feature. They are a no longer reserved, but also no longer legal in identifiers. **GEDCOM 5.5.5 solves many pointer problems simply by restricting identifiers to alphanumeric characters**.

References

GEDCOM Identifiers: Length

XREF:FAM:= **{Size=3:22}**

A pointer to, or a cross-reference identifier of, a <<FAM_GROUP_RECORD>> (FAM record).

XREF:INDI:= **{Size=3:22}**

A pointer to, or a cross-reference identifier of, an <<INDIVIDUAL_RECORD>> (INDI record).

XREF:NOTE:= **{Size=3:22}**

A pointer to, or a cross-reference identifier of, a <<NOTE_RECORD>> (NOTE record).

XREF:OBJE:= **{Size=3:22}**

A pointer to, or a cross-reference identifier of, a <<MULTIMEDIA_RECORD>> (OBJE record).

XREF:REPO:= **{Size=3:22}**

A pointer to, or a cross-reference identifier of, a <<REPOSITORY_RECORD>> (REPO record).

XREF:SOUR:= **{Size=3:22}**

A pointer to, or a cross-reference identifier of, a <<SOURCE_RECORD>> (SOUR record).

XREF:SUBM:= **{Size=3:22}**

A pointer to, or a cross-reference identifier of, a <<SUBMITTER_RECORD>> (SUBM record).

YEAR:= **{Size=3:4}**

A numeric representation of the calendar year in which an event occurred.

One and two-digit years are illegal. Years 1 through 99 must be padded out to at least 3 digits by using leading zeroes.

Three Digits Demand is Obsolete

The demand to use at least 3 digits for the year is obsolete. This demand was put in to avoid possible confusion between years and days as entered into early genealogy applications.
Date entry in genealogy applications continues to require developer and user attention, particularly in locales still using antilogical date formats. However, the fixed order date formats used in GEDOM do not allow confusion between days and years.

The GEDCOM File

File Name

The GEDCOM file is normally created in a user directory. The filename extension is `.GED` or `.ged`; the casing does not really matter, but by convention the filename extension is either all-uppercase or all-lowercase.
Macintosh filenames do not use file extensions.

Multi-Volume GEDCOM Files

GEDCOM 5.0 (1991) through 5.5.1 support multi-volume GEDCOM files. A multi-volume file is a file split into multiple files (the volumes); each of the volumes should fit onto a single floppy disk or diskette, even if the original file does not. This feature was never necessary, as multi-volume archivers were widely available.

In practice, all the Multi-volume GEDCOM support accomplished is needlessly complicating the reading of GEDCOM files; a GEDCOM 5.5.1 reader must not assume that the end of a file is the end of the GEDCOM file; the GEDCOM file may continue on another volume (the next file in the multi-volume series).
The *GEDCOM 5.5.1 Annotated Edition* deprecated this misfeature and the GEDCOM 5.5.5 specification obsoletes it; it is no longer allowed to break a GEDCOM file into multiple files. This simplifies GEDCOM readers; the end of the GEDCOM file is the end of the GEDCOM file.

User-Defined Tags

Use of user-defined GEDCOM tags is not encouraged. Applications requiring the use of nonstandard tags must define them with a leading underscore so that they will not conflict with future GEDCOM standard tags. The meaning and interpretation of user-defined tags is system-dependant. The system that defined the tags is identified in HEAD.DEST.

The system in HEAD.DEST is often, but not always, the same system as that in HEAD.SOUR; applications can and do generate GEDCOM files intended for particular third-party systems.

Escape Sequences

Use of GEDCOM escape sequences is not recommended. The only use of escape sequences in the current Lineage-Linked Form is the use of calendar escape sequences, to specify a calendar and a date in single line value.

References

Multi-volume GEDCOM files

Sample Lineage-Linked GEDCOM File

The example below is a sample GEDCOM file of genealogical information about three individuals who are members of the same family — father, mother, and child. In the example, "Joe/Williams/" is the value specified by the tag NAME under the INDI tag for the record (@I3@). Other values in other lines, such as the birth date and place, provide additional information about Joe Williams. The value (@F1@) specified by the FAMC tag is a pointer to the <<FAM_GROUP_RECORD>> (@F1@) of which Joe Williams is a child. Included also in this GEDCOM file example are three other record types: a source record, a submitter record, and a repository record. These records are pointed to from within other records in the GEDCOM file. This shows how pointer values can be used in creating a Lineage-Linked GEDCOM file.

Example:

```
0 HEAD
1 GEDC
2 VERS 5.5.5
2 FORM LINEAGE-LINKED
3 VERS 5.5.5
1 CHAR UTF-8
1 SOUR GS
2 NAME GEDCOM Specification
2 VERS 5.5.5
2 CORP gedcom.org
3 ADDR
4 CITY LEIDEN
3 WWW www.gedcom.org
1 DATE 2 Oct 2019
2 TIME 0:00:00
1 FILE 555Sample.ged
1 LANG English
1 SUBM @U1@
0 @U1@ SUBM
1 NAME Reldon Poulson
1 ADDR
2 ADR1 1900 43rd Street West
2 CITY Billings
2 STAE Montana
2 POST 68051
2 CTRY United States of America
1 PHON +1 (406) 555-1232
0 @I1@ INDI
1 NAME Robert Eugene /Williams/
2 SURN Williams
2 GIVN Robert Eugene
1 SEX M
1 BIRT
2 DATE 2 Oct 1822
2 PLAC Weston, Madison, Connecticut, United States of America
2 SOUR @S1@
3 PAGE Sec. 2, p. 45
1 DEAT
2 DATE 14 Apr 1905
2 PLAC Stamford, Fairfield, Connecticut, United States of America
```

1 BURI
2 PLAC Spring Hill Cemetery, Stamford, Fairfield, Connecticut, United States of America
1 FAMS @F1@
1 FAMS @F2@
1 RESI
2 DATE from 1900 to 1905
0 @I2@ INDI
1 NAME Mary Ann /Wilson/
2 SURN Wilson
2 GIVN Mary Ann
1 SEX F
1 BIRT
2 DATE BEF 1828
2 PLAC Connecticut, United States of America
1 FAMS @F1@
0 @I3@ INDI
1 NAME Joe /Williams/
2 SURN Williams
2 GIVN Joe
1 SEX M
1 BIRT
2 DATE 11 Jun 1861
2 PLAC Idaho Falls, Bonneville, Idaho, United States of America
1 FAMC @F1@
1 FAMC @F2@
2 PEDI adopted
1 ADOP
2 DATE 16 Mar 1864
0 @F1@ FAM
1 HUSB @I1@
1 WIFE @I2@
1 CHIL @I3@
1 MARR
2 DATE Dec 1859
2 PLAC Rapid City, Pennington, South Dakota, United States of America
0 @F2@ FAM
1 HUSB @I1@
1 CHIL @I3@
0 @S1@ SOUR
1 DATA
2 EVEN BIRT, DEAT, MARR
3 DATE FROM Jan 1820 TO DEC 1825
3 PLAC Madison, Connecticut, United States of America
2 AGNC Madison County Court
1 TITL Madison County Birth, Death, and Marriage Records
1 ABBR Madison BMD Records
1 REPO @R1@
2 CALN 13B-1234.01
0 @R1@ REPO
1 NAME Family History Library
1 ADDR
2 ADR1 35 N West Temple Street
2 CITY Salt Lake City
2 STAE Utah
2 POST 84150
2 CTRY United States of America
0 TRLR

Chapter 3 Using Character Sets in GEDCOM

Introduction

GEDCOM was created to exchange genealogical data in multiple languages, so it needs to support the characters used by those languages. Back in the 20th century of the Common Era, when GEDOM was still focussed on Western languages, FamilySearch wisely decided to disallow code pages and use ANSEL instead. GEDCOM 5.3 (1993 CE) specification and later allow Unicode in support of non-Western languages, with ANSEL remaining the default. The GEDCOM 5.4 (1995 CE), GEDCOM 5.5 and GEDCOM 5.5.1 (1999 CE) specification all state that Unicode will eventually replace ANSEL as the standard character set for GEDCOM.
In the twenty years that passed since the release of GEDCOM 5.5.1 the industry has switched to Unicode. The *GEDCOM 5.5.1 Annotated Edition* (2018 CE) already states all genealogy applications should be Unicode-based and always export to Unicode. GEDCOM 5.5.5 completes the transition to Unicode.
GEDCOM 5.5.5 does not allow ANSEL. GEDCOM 5.5.5 demands Unicode
Systems should continue to support the ANSEL encoding for importing GEDCOM 5.5.1 files.

GEDCOM Character Sets and Encoding

GEDCOM 5.5.5 has just one legal character set: Unicode. Earlier version of GEDCOM supports three character sets: ASCII, ANSEL and Unicode.
ASCII and ANSEL are characters sets with just one encoding. Unicode has multiple encodings, GEDCOM supports the two most important ones, UTF-8 and UTF-16.
Thus, GEDCOM supports four character encodings: ASCII, ANSEL, UTF-8 and UTF-16. The `HEAD.CHAR` line values specifying these encodings are `ASCII`, `ANSEL`, `UTF-8`, and `UNICODE`.

UTF-16: `UNICODE`

Previous GEDCOM specifications erroneously refer to UTF-16 as UNICODE (all-caps), and introduced the `HEAD.CHAR` line value `UNICODE` for it.
For compatibility's sake, the `HEAD.CHAR` value for UTF-16 remains `UNICODE`. The `UNICODE` value must be understood as meaning UTF-16, and *must not* be used for any other Unicode encoding. UTF-8 GEDCOM files *must* use the `HEAD.CHAR` value `UTF-8`.

Legal Encodings

GEDCOM 5.5.5 has just two legal encodings, UTF-8 and UTF-16. Earlier versions of GEDCOM support four legal encodings: ASCII, ANSEL, UTF-8 and UTF-16.
Historically, many system have created not-really-GEDCOM files using illegal character sets and encodings. Developers want users to be able to import third-party GEDCOM and not-really-GEDCOM files, so many GEDCOM 5.5 and 5.5.1 readers support the use of these illegal character sets and encodings. That well-intentioned flexibility has a serious deleterious effect. By obviating the need for the developers of the offending third-party product to fix their GEDCOM output, those too-flexible GEDCOM readers keep the malpractice and the problems its creates alive.
GEDCOM 5.5.5 ends this malpractice without breaking existing software.

GEDCOM 5.5.5 does not tolerate the use of illegal character sets or encodings.

- GEDCOM 5.5.5 writers *must not* use anything but the legal encodings.
- GEDCOM 5.5.5 readers *must not* accept anything but the legal encodings.
- GEDCOM 5.5.5 readers *must* reject all illegal encodings.

Systems that currently accept illegal character sets and encodings in GEDCOM 5.5.1 or earlier may continue to do so for the sake of compatibility, but only in GEDCOM 5.5.1 and earlier. Developers that want to continue using illegal character sets and encodings, must continue to do

so using GEDCOM 5.5.1 or earlier.

Support

All systems supporting GEDCOM 5.5.5 (or later) should default GEDCOM export to GEDCOM 5.5.5.
All GEDCOM 5.5.5 export *must* use the Unicode character set. All Western systems should default their GEDCOM export to UTF-8 encoding. All Eastern systems should default their GEDCOM export to UTF-16 encoding. Users should be able to override that default by setting a file property.

Export of Unicode data to ASCII or ANSEL would be practically sure to lose information. That is why GEDCOM 5.5.5 does not allow it. Unicode-based systems that already support export to ANSEL-encoded GEDCOM 5.5.1 may and should continue to offer that option, for the sake of compatibility with the user's existing practices. However, export to non-Unicode encodings should not be offered via the regular GEDCOM export dialog box, but only as an advanced option.

Real World UTF-8 only Export

Several applications were offering Unicode-only GEDCOM export before GEDCOM 5.5.5 already. RootsMagic has offered nothing but UTF-8 GEDCOM export since it became Unicode-based with RootsMagic 4 in 2008 CE. Software MacKiev Family Tree Maker changed to UTF-8 only export starting with Family Tree Maker 2017, and Ahnenblatt changed to UTF-8 only export with Ahnenblatt 3.0, in 2019 CE.

Systems should be able to import all four legal encodings.
UTF-8 and UTF-16 are the preferred encodings of GEDCOM files. GEDCOM 5.5.5 systems *must not* create ANSEL GEDCOM files, but import of ANSEL GEDCOM 5.5.1 files must be supported for backward compatibility.

Code Page-based Systems

GEDCOM 5.5.5 demands a Unicode-based system.
Code page-based systems should no longer be in use. Users of a code page-based system are strongly recommended to stop using it and upgrade to one of the many Unicode-based system.

Code page-based systems *must not* claim or attempt to support GEDCOM 5.5.5, but must continue to use GEDCOM 5.5.1 or earlier.
The rule for code page-based systems has always been to export to GEDCOM using ANSEL, but it is recommended that even code-page based system now default their GEDCOM export to using UTF-8.

ANSEL

ANSEL (American National Standard for Extended Latin Alphabet Coded Character Set for Bibliographic Use, Z39.47-1985) is an 8-bit character set. The official specification can be obtained from the American National Standards Institute (ANSI).
ANSEL was *the* American Library Association character set, and was used in library systems worldwide, including the MARC (Machine-Readable Catalog) format. Those systems have now transitioned to Unicode.

ANSEL is a superset of ASCII. ANSEL extends ASCII with a some useful characters and non-spacing character modifiers (diacritics), resulting in a single character set for Western languages (instead of a confusing mess of incompatible code pages).

ANSEL versus “ANSEL”

FamilySearch has extended ANSEL and kept calling the resulting character set ANSEL, in direct

violation of the ANSEL specification. FamilySearch also kept changing what's in their “ANSEL”; different versions of GEDCOM have different “ANSEL” tables. GEDCOM 5.5.5 provides a consolidated “ANSEL” table in Appendix C ANSEL Character Set. It is strongly recommended that genealogy software developers use this single table for all GEDCOM versions they support.

Previous versions of GEDCOM stated that ANSEL is the preferred character encoding for GEDCOM files. GEDCOM 5.5.1 (1999 CE) already stated that this would shortly change to UTF-8 and UTF-16. GEDCOM 5.5.1 remained the last version of GEDCOM for almost two decades and during that time, the industry has switched to Unicode as the preferred character set and UTF-8 as the preferred encoding. GEDCOM 5.5.5 merely makes that official.

Starting with GEDCOM 5.5.5, Unicode is the preferred character set for GEDCOM files. Systems can use either the UTF-8 or UTF-16 encoding.
Use of ANSEL remains legal, but is deprecated now.

ASCII

The American Standard Code for Information Interchange (ASCII) is one of the oldest character set standards (1963 CE). ASCII is an USA-centric character set. Most Western languages need characters not supported by ASCII, which is why ANSEL was the preferred character set for GEDCOM files. Many languages need characters not supported by ANSEL, which is why Unicode is the preferred character set for GEDCOM files. ANSEL and Unicode are both supersets of ASCII.

Previous versions of GEDCOM referred to in various incorrect and confused ways, such as “ASCII (USA Version)”; There are no country-specific versions of ASCII. There is just ASCII, which is American. Many country-specific code pages are based on ASCII, but *are not ASCII*.

Previous versions of GEDCOM also referred to ASCII as “8-bit ASCII”, and that is a contradiction in terms. There is no such thing as 8-bit ASCII. ASCII is a 7-bit character set.

Unicode

Unicode and ISO 10646 are not exactly the same thing, but do define the same character set. It is a rather persistent myth that Unicode is a 16-bit character set - it is not. Unicode is a 21-bit character set that currently supports more than a quarter million different characters.
Unicode *is* a superset of ASCII and the many 8-bit code pages it replaces.

Unicode is a single character set that supports quite a few different encodings. GEDCOM allows only the two most important ones, UTF-8 and UTF-16. Support for UTF-8 and UTF-16 is built into all modern operating systems; developers need not and should not create their own conversions.

Developer who ignore the above advice and try to make their own conversions anyway should take particular care to make sure that their conversion from the UTF-16 encoding used by the operating system to UTF-8 is indeed a conversion to UTF-8 and not a conversion to CESU-8. GEDCOM readers that take advantage of operating system routines are not unlikely to reject ostensible UTF-8 GEDCOM files that are actually CESU-8 encoded because of illegal code sequences. This is a security measure.

To understand Unicode and the supported encodings, it is essential to understand the difference between characters, code points and code units.
Very briefly: A code point is a particular value within a character set. A single character may consist of multiple code points.
A code unit is the smallest size used in a particular encoding of a character set; UTF-8 has 8-bit code units, UTF-16 has 16-bit code units. An encoding may require multiple code units to represent one code point.

Byte Order Mark

GEDCOM files are text files, so UTF-8 GEDCOM files are UTF-8 text files. Per Unicode rules,

starting the text file with Byte Order Mark (BOM) is mandatory for UTF-16 text files, but optional for UTF-8 text files.

Starting UTF-8 GEDCOM files with a BOM makes sure that text editors will recognise the GEDCOM file as UTF-8 encoded, so that they do not misrecognise the encoding and mess up the file. Starting UTF-8 GEDCOM files with a BOM prevents really old genealogy applications, that do not understand UTF-8, from recognising the GEDCOM file as a GEDOM file. Many old applications do not check the encoding, and will try to import the GEDCOM file anyway, despite not supporting the encoding used (and not supporting Unicode at all), resulting in significantly mangled data and data loss. Including a BOM prevents these old applications from doing so.

The *GEDCOM 5.5.1 Annotated Edition* strongly recommends that UTF-8 GEDCOM files start with a BOM. **GEDCOM 5.5.5 specifies that UTF-8 GEDCOM files *must* start with a Byte Order Mark (BOM).** This rule aligns with existing practice, as started with FamilySearch PAF 5.0 (2000 CE).

Using Character Encodings

Previous versions of GEDCOM suggested that it might be possible to change character sets in the middle of a GEDCOM file - it is not. That notion was abandoned, but even GEDCOM 5.5.1 still contained confusing remnants of that notion. Each GEDCOM file uses just one encoding throughout.
Previous versions of GEDCOM suggested that, regardless of whatever encoding is used for the GEDCOM file, the GEDCOM header should always be ANSEL-encoded. That is wrong. The entire GEDCOM file, including the GEDCOM header, always uses just one encoding.
The encoding used by a GEDCOM file is specified in the GEDCOM header through the mandatory `HEAD.CHAR` record.

The following partial GEDCOM header provides an example:

```
0 HEAD
1 SOUR PAF
2 VERS 5.2.18
1 DEST PAF
1 CHAR UTF-8
...
```

Detecting Character Encoding

The GEDCOM specification provides practically no information about detecting GEDCOM files and the encoding used by GEDCOM files.

References

- The Absolute Minimum Every Software Developer Absolutely, Positively Must Know About Unicode and Character Sets (No Excuses!)
- 0 HEAD Value
- GEDCOM & FTW TEXT Magic
- GEDCOM Magic Values
- GEDCOM Character Encodings

Unicode Books

The Unicode Standard has seen several editions since the publication of GEDCOM 5.5.1

back in 1999. It is a reference, not very suitable for learning about Unicode.
The Unicode Consortium web site offers many helpful resources, including an introduction to Unicode and a FAQ.

Two useful books are:

- *Unicode Demystified; A Practical Programmer's Guide to the Encoding Standard* by Richard Gillam (2003, Addison-Wesley, pp. 853, ISBN 0-201-70052-2)
- *Developing International Software, Second Edition* by Dr. International (2003, Microsoft Press, pp. 1060, ISBN 0-6536-1583-7)
  This is the revised edition of *Developing International Software for Windows 95 and Windows NT* by Nadine Kano (1995, Microsoft Press, pp. 743, ISBN 1-55615-840-8).

Chapter 4 GEDCOM System Identifiers

System Identifier Registration

Each system that reads or writes GEDCOM files must have a system identifier to identify that system by. That system identifier must be unique, so that there can be no confusion about which system is meant.
The obvious way to ensure uniqueness is through a centrally maintained registry of system identifiers. FamilySearch used to have a registration process, but there is no registry today or registration process now. FamilySearch silently abandoned GEDCOM around 2000 CE. FamilySearch does not provide a list of system identifiers registered before 2000 CE, and most current genealogy applications were created after 2000 CE.

GEDCOM Header

System identifiers are used in the GEDCOM header, to identify the system that created the GEDCOM file, and the system that the GEDCOM file is for. The HEAD.SOUR value identifies the system that created the GEDCOM file, the HEAD.DEST value identifies the system that it is for. It is the HEAD.DEST value, not the HEAD.SOUR value, that identifies the meaning and interpretation of any GEDCOM extensions used in the GEDCOM file.

A typical GEDCOM file isn't created for a specific other system, but does contain GEDCOM extensions specific to the system that created it; that's why the HEAD.DEST value must default to system's own system identifier. Thus, in a typical GEDCOM file, the HEAD.SOUR value and the HEAD.DEST value are identical.

Maximum System Identifier Length

The maximum length of a system identifier is 20 code units, as stated in the <SYSTEM_ID> definition.
Some confusion was created by the GEDCOM 5.5.1 chapter *GEDCOM Product Registration*, which stated that system identifiers may be up to 40 characters long. The actual definition takes precedence over statements in chapters, and that chapter is gone now.

Case-Insensitive

System identifiers aren't controlled line values and should be case-sensitive, but long existing practice is to treat system identifiers as case-insensitive. Therefore, GEDCOM System Identifiers are case-insensitive.
Developers are still advised to pick one particular casing and stick with it, for consistency's sake. Use of mixed case is preferred, but abbreviations should be uppercase.

Spaces

It is legal to use spaces in system identifiers. Use of spaces within system identifiers has always been allowed, despite earlier versions of GEDCOM saying otherwise, as FamilySearch themselves has been using system identifier that contain spaces for decades.

Non-ASCII

The use of non-ASCII characters in the system identifier is legal, and *not* discouraged.
Systems reading GEDCOM 5.5.5 files know the character set and encoding used in the header as soon as they've processed the Byte Order Mark (BOM).

Digits

System identifiers may contain digits. However, system identifiers *must not* contain version numbers.

Version Numbers

System identifiers *must not* contain version numbers. Different versions of the same system must use the same system identifier. The version number of the system can and must be specified in HEAD.SOUR.VERS.

Editions

Some products are available in multiple editions, such as a Lite Edition or a Pro Edition. System identifiers *must not* indicate an edition. Different editions of the same system must use the same system identifier. The edition may be included in the product name HEAD.SOUR.NAME.

Platforms

The system identifier *should not* include the platform name (such as “Windows”).
Multiplatform systems *must not* include the platform name, but must use the GEDCOM reading and writing code, and the same system identifiers on all platforms.

Multiple Identifiers

Each system must be known by just one system identifier. Use of multiple, different identifiers for a single system is not allowed.

Some genealogy software developers have sold the same product under different names in different language regions, and then used a different name for each regional version. This is wrong.
The system identifier must not change merely because the user interface got translated.

Invalid System Identifiers

Systems creating GEDCOM 5.5.5 files *must* use a valid system identifier. Systems reading GEDCOM 5.5.5 files *must* refuse invalid system identifiers, even if they accept these when used in GEDCOM 5.5.1 files.
GEDCOM 5.5.5 readers *must* reject system identifiers that are too long, contain product version numbers or product editions.
GEDCOM 5.5.5 readers *should* also reject all otherwise valid system identifiers that are part of a collection of multiple system identifiers for a single system.

Nonsense Values

GEDCOM specifications before the GEDCOM 5.5.1 Annotated Edition (2018 CE) failed to clearly specify the usage of the HEAD.DEST value. Many GEDCOM 5.5 and 5.5.1 files contain nonsense values such as `ANY`, `GEDCOM55`, and `Other`, despite the fact that the specification demands that the HEAD.DEST value be a system identifier.

GEDCOM 5.5.5 does not tolerate this malpractice. GEDCOM 5.5.5 demands that the HEAD.DEST value is a real system identifier. Known nonsense values must not be used as system identifiers.

Known Nonsense Values

The following are known nonsense values for the system identifier that *must not* be used. GEDCOM 5.5.5 readers that encounter any of these values instead of an actual system identifier, *must* reject the file as not-GEDCOM, specifically as not even having a valid GEDCOM header.

known nonsense values
`ANY`
`GED55`
`GEDCOM`
`GEDCOM55`
`Other`

The above table of known nonsense values may be expanded as additional nonsense values are identified. Developers are encouraged to take full advantage of the table published in the latest GEDCOM version.
Obviously, none of the known nonsense values may be adopted as the system identifier for any new or existing system.

Quick summary of System Identifier Rules

- A system identifier must be unique
- System identifiers are case-insensitive.
- Spaces are allowed
- Digits are allowed
- Non-ASCII characters are allowed
- The maximum length is 20 code units
- Same, single system identifier for all versions and platforms
- Version numbers or version indicators are not allowed
- The system identifier must not include any platform indication
- Known nonsense values are illegal

GEDCOM Reader Rules

- Treat system identifiers as case-insensitive
- Accept spaces in system identifiers
- Accept non-ASCII characters (i.e. figure out the character encoding first)
- Issue a fatal error (not even a correct GEDCOM header!) when the system identifier is too long
- Reject known nonsense value with a fatal error: invalid GEDCOM header

GEDCOM Validator Best Practice

- Allow spaces, inform that spaces were officially illegal, never really illegal
- Warn against usage of non-ASCII characters in pre-GEDCOM 5.5.5 files only
- Issue a fatal error (not even a correct GEDCOM header!) when the system identifier is too long
- Issue a fatal error (not even a correct GEDCOM header!) when encountering known nonsense values
- Upon encountering an ALL-CAPS system identifier, suggest the use of lower case for readability
- Try to detect version numbers in system identifiers, and issue a strong warning against it

References

- GEDCOM System Identifiers
- GEDCOM SOUR and DEST